

Utilisation Efficace et Responsable de l'IA Générative

en Développement Logiciel

WEB-2026 – HEIG-VD

Clarisse Fleurimont – Mai 2026



Plan de la session

- **Introduction** – C'est quoi l'IA générative ?
- **Bonnes pratiques** – Comment l'utiliser correctement ?
- **Travail de Master** – Ressource à disposition
- **Démonstration Live** – Claude Code sur Tetris III
- **Questions & Discussion**

Durée : 45 minutes

Qu'est-ce que l'IA Générative ?

Des outils capables de :

- Générer du code à partir de descriptions en langage naturel
- Compléter du code existant (autocomplétion intelligente)
- Expliquer et documenter du code
- Détecter des bugs et proposer des corrections
- Générer des tests unitaires

Exemples : GitHub Copilot, Claude Code, Cursor, ...

Le Principe Fondamental

L'IA propose,
la personne valide et décide

L'IA est un **outil**, pas un remplaçant.

Méthodologie Human-in-the-Loop

1. Proposer

L'IA génère du code

2. Vérifier

Relecture critique

3. Évaluer

Trop de corrections ?

4. Accepter

Valider le code

5. Tester

Exécuter les tests

6. Intégrer

Code validé !

⚠ Si vous ne comprenez pas le code généré, ne l'intégrez pas !

Attentes Réalistes

Statistiques sur le code généré par IA

- Code IA contient **1.7x plus de bugs** que code humain (étude GitClear 2024)
- **62% des snippets** contiennent des failles de sécurité (Stanford 2023)
- Gain de productivité réel : **20-30%**, pas 10x

✓ L'IA excelle pour...

- Boilerplate code
- Tests unitaires simples
- Documentation

× L'IA échoue souvent sur...

- Logique métier complexe
- Sécurité et validation
- Architecture système

Efficacité de l'IA par Type de Tâche

Type de tâche	Efficacité	Supervision	Risque
Génération de tests	Élevée	Faible	Faible
Refactoring localisé	Élevée	Faible	Faible
Documentation	Élevée	Faible	Faible
Configuration	Moyenne	Moyenne	Moyen
Refactoring multi-fichiers	Moyenne	Élevée	Élevé
Code avec sécurité	Faible	Très élevée	Critique

Source : Travail de Master – Modernisation d'une architecture legacy

Bonnes Pratiques : À Faire

- ✓ **Toujours relire le code généré** – Ne jamais faire confiance aveuglément
- ✓ **Comprendre avant d'accepter** – Demandez des explications si nécessaire
- ✓ **Tester systématiquement** – Le code peut compiler mais ne pas fonctionner
- ✓ **Donner du contexte** – Plus le prompt est précis, meilleur est le résultat
- ✓ **Itérer** – Affiner progressivement plutôt que tout demander d'un coup

Bonnes Pratiques : À Éviter

- ✗ **Copier-coller sans comprendre**
- ✗ **Faire confiance aux restructurations automatiques** sans vérification
- ✗ **Utiliser l'IA pour du code critique** sans revue approfondie
- ✗ **Ignorer les messages d'erreur** et juste demander “corrige”
- ✗ **Penser que l'IA comprend votre intention** – elle ne fait que prédire

L'IA peut introduire des failles de sécurité (injection, XSS, etc.)

Risques de Sécurité : Exemples Concrets

L'IA génère souvent du code **fonctionnel mais vulnérable** :

```
// ❌ XSS - innerHTML non échappé  
chat.innerHTML += `<p>${msg}</p>`;
```

```
// ❌ SQL Injection  
db.query(`SELECT * FROM users  
WHERE id = ${userId}`);
```

```
// ❌ ReDoS - regex catastrophique  
const valid = /^(.+)+$/ .test(input);
```

```
// ❌ Prototype pollution  
Object.assign(target, userInput);
```



Règle d'or : Toujours valider les entrées utilisateur vous-même

Écrire de Bons Prompts

Un bon prompt contient :

- **Le contexte** – Projet, fichier, technologie
- **L'objectif** – Ce que vous voulez accomplir
- **Les contraintes** – Style, patterns
- **Les critères de succès** – Comment valider

Exemple :

```
Dans messages.js, implémente encode() du MessageCodec.  
Elle doit convertir un Message en JSON : {type: "NomClasse", data: ...}  
Utilise MessageCodec.types pour obtenir le nom de la classe.
```

Mon Travail de Master

Modernisation d'une architecture legacy

Migration modulithique, Keycloak et assistance par IA – MSE 2026

Contexte

- Plateforme médicale HeartCare
- 50'000+ lignes de code Java
- Utilisation de Claude Code
- Agents spécialisés par domaine

Résultats

- 30-40% gain de temps (certaines tâches)
- 5/6 agents IA opérationnels
- Couverture tests : 34% → 68%
- Traçabilité IA/humain : non résolue



Disponible : ressource/FleurimontClarisse_presentation_MSE_TM_SA25.pdf

Enseignements Clés du TM

✓ Points forts de l'IA

- Génération de tests
- Documentation
- Analyse de code
- Refactoring localisé

× Limitations

- Restructuration architecture
- Déplacement de fichiers
- Vision globale du projet
- Contexte limité (tokens)

“L'IA est un outil, pas un remplaçant”

Démonstration Interactive (25 min)

Projet : Tetris III – WebSocket • Fichier : `src/messages.js`

3 Segments de démonstration :

1. Succès (7 min)

Prompt précis → Code correct du 1er coup

2. Échec (10 min)

Prompt vague → Itération → Succès

3. Sécurité (8 min)

Code fonctionnel mais vulnérable

Message clé : L'échec et l'itération font partie du workflow normal

Démo : Segment 1 - Succès

Fonction : encode(message)

Prompt :

"Dans messages.js, implémente encode() du MessageCodec.
Elle doit convertir un Message en JSON : {type: "NomClasse", data: ...}
Utilise MessageCodec.types pour obtenir le nom de la classe."

- Contexte clair (fichier, classe, format)
- Contrainte explicite (utiliser MessageCodec.types)
- Résultat attendu défini

✓ **Cas idéal** : Tests passent du premier coup

Démo : Segment 2 - Échec & Itération

Fonction : `decode(string)`

✗ Prompt vague

“Implémente decode qui fait l'inverse de encode”

✓ Prompt amélioré

“decode() doit : 1) Parser JSON, 2) Extraire type, 3) Instancier...”

Workflow réaliste :

1. Tests échouent → 2. Lire l'erreur → 3. Reformuler le prompt → 4. Réessayer

L'itération est normale, pas un signe d'échec !

Démo : Segment 3 - Risque Sécurité

Prompt : “Ajoute une validation des messages”

L'IA peut générer :

```
// Vulnérable au ReDoS
if (/^(.+)+$/ .test(input)) { ... }

// Manque try/catch
const obj = JSON.parse(input);

// Prototype pollution possible
if (obj.type && obj.data) { ... }
```



Le code compile et fonctionne, mais n'est pas sûr

Votre responsabilité : revue de sécurité systématique

Points à Observer

- **Contexte initial** – Comment je présente le projet à Claude
- **Prompt précis** – Structure de la demande
- **Vérification** – Est-ce que je comprends le code ?
- **Tests** – npm test pour valider
- **Itération** – Correction si erreur (avec le message d'erreur !)

Questions à vous poser : “Qu’est-ce qui pourrait mal tourner ?” • “Comment vérifier que c’est correct ?”

Exercice : Trouve le Bug

3 snippets de code générés par IA

Chacun contient un problème. Lequel ?

Snippet 1

Faible de sécurité

Snippet 2

Bug de logique

Snippet 3

Anti-pattern

Format : Je montre le code → Vous identifiez le problème → On discute

Snippet 1 : Affichage de message

```
// Code généré pour afficher un message dans le chat
function displayMessage(msg) {
  document.getElementById('chat').innerHTML += `<p>${msg}</p>`;
}
```

Quel est le problème ?

Indice : Que se passe-t-il si msg = <script>alert('XSS')</script> ?

Snippet 2 : Récupérer les derniers messages

```
// Code généré pour récupérer les N derniers messages
function getLastNMessages(messages, n) {
  return messages.slice(messages.length - n - 1);
}
```

Quel est le problème ?

Indice : Si `messages.length = 10` et `n = 3`, combien d'éléments ?

Snippet 3 : Charger les utilisateurs

```
// Code généré pour charger les utilisateurs avec profils
async function getAllUsersWithProfiles() {
  const users = await db.getUsers();
  for (const user of users) {
    user.profile = await db.getProfile(user.id);
  }
  return users;
}
```

Quel est le problème ?

Indice : Combien de requêtes pour 100 utilisateurs ?

Quand Utiliser l'IA ?

Arbre de décision

✓ Utiliser l'IA

- Boilerplate, tests unitaires
- Documentation, commentaires
- Refactoring simple
- Exploration d'API
- Prototypage rapide

× Coder soi-même

- Logique métier critique
- Code avec authentification
- Manipulation de données sensibles
- Architecture système
- Code dont on ne comprend pas le résultat

 **Règle simple : Si vous ne pouvez pas vérifier, ne l'utilisez pas**

Aspects Éthiques et Légaux

- **Propriété intellectuelle** – Le code généré peut ressembler à du code existant
- **Confidentialité** – Ne pas partager de code confidentiel avec des outils en ligne
- **Responsabilité** – Vous restez responsable du code que vous intégrez
- **Citation académique** – Mentionnez l'utilisation de l'IA dans vos travaux

Dans un contexte professionnel, vérifiez les politiques de votre entreprise !

Ressources

Pendant le cours

- **Document de support** : PDF
- **Quick Reference Card** : aide-mémoire A4

Auto-étude

- **Exercices Spot-the-Bug** : 10 exercices
- **Exemples de prompts** : bons/mauvais
- **Questionnaire** : auto-évaluation

Documentation

- **Travail de Master** : PDF
- docs.anthropic.com/claude-code
- docs.github.com/copilot

Tous les outils ont une version gratuite pour pratiquer !

Questions ?

Rappel : L'IA est un **outil puissant**,
mais la **supervision humaine** reste indispensable.

Bonne pratique avec l'IA générative !